Encapsulamiento: que el objeto se haga cargo de sí mismo

El problema

En la primera clase escribimos una clase Auto y quedó una nota al pie: ese código estaba mal a propósito y lo íbamos a retomar en la unidad 2. Es hoy.

```
public class Auto {
    String marca;
    String modelo;
    int anio;
}
```

Con los atributos declarados así, cualquier parte del programa puede escribir lo que quiera adentro de un auto:

```
Auto auto1 = new Auto();
auto1.marca = null;
auto1.modelo = "";
auto1.anio = -3000;
```

Ese código compila y corre sin que nadie proteste. Queda en memoria un Auto sin marca, sin modelo y fabricado en el año 3000 antes de Cristo. El programa no falla acá: va a fallar mucho después, cuando alguien invoque mostrarDetalles(). El error se causó en un lugar y se manifiesta en otro, que es la peor clase de error para encontrar.

Hay dos problemas distintos. El primero es que la clase Auto sabe perfectamente qué es un auto válido y no tiene ninguna forma de hacerlo cumplir. El segundo es que el objeto se crea vacío con new Auto() y recién después se le completan los datos de a uno: si el programador se olvida de una línea, el auto queda a medio hacer para siempre.

La pregunta de esta clase es quién es responsable de que los datos de un objeto tengan sentido. En programación procedural la respuesta era: el que los usa. En objetos la respuesta es otra, y cambia por completo cómo se escribe una clase.

Encapsulamiento y ocultamiento de la información

El encapsulamiento es la capacidad de reunir en una sola entidad, la clase, los datos de un objeto y las operaciones que trabajan sobre esos datos. Esa parte ya la veníamos usando sin nombrarla: cada vez que escribimos un método adentro de la clase, en lugar de una función suelta que recibe el objeto como parámetro, estamos encapsulando.

Lo que se agrega hoy es la otra mitad de la idea: el ocultamiento de la información. Juntar datos y operaciones sirve de poco si los datos siguen estando al alcance de cualquiera. Ocultar significa que el estado interno queda fuera del alcance del resto del programa, y que lo único visible desde afuera es un conjunto de operaciones públicas que funcionan como contrato: esto es lo que se le puede pedir a este objeto, y nada más. Es la lógica de un cajero automático,

donde nadie abre la máquina para sacar billetes de las gavetas: se pide una operación por la pantalla, y el cajero decide si se puede hacer y es el único que toca el dinero.

La idea que ordena todo: el estado válido

Cada clase, aunque no lo escriba en ningún lado, hace una promesa sobre sus objetos. La clase Auto promete que un auto tiene una marca, un modelo y un año razonable. Una billetera virtual promete que su saldo nunca es negativo. Esa promesa es lo que hace que el objeto sea confiable: si se cumple siempre, el resto del programa puede usarlo sin desconfiar.

Todo lo que sigue son las cuatro herramientas que da Java para sostener esa promesa, y conviene leerlas como partes de una sola estrategia y no como cuatro temas sueltos: private cierra la puerta para que nadie la rompa desde afuera, el constructor garantiza que el objeto nazca cumpliéndola, los métodos públicos la mantienen mientras el objeto vive y los getters permiten mirar el estado sin poder modificarlo.

Modificadores de acceso

Java controla el ocultamiento con los modificadores de acceso, que determinan desde dónde se puede usar un atributo o un método.

Modificador	Alcance	Descripción
public	Máximo	Accesible desde cualquier clase.
protected	Protegido	Accesible desde el mismo paquete y desde las subclases.
default	Paquete	Lo que queda si no se escribe ningún modificador. Accesible solo desde las clases del mismo paquete.
private	Mínimo	Accesible únicamente desde adentro de la propia clase.

Por ahora vamos a usar dos: private para los atributos y public para las operaciones que la clase ofrece. Los otros dos van a tener sentido cuando veamos herencia y paquetes, en la unidad 3. La regla práctica casi no tiene excepciones: los atributos van private.

El constructor

Poner los atributos en private resuelve la mitad del problema. La otra mitad era el objeto a medio construir, y de eso se ocupa el constructor: un método especial cuya única finalidad es dejar el objeto en un estado válido en el momento mismo de su creación. Se llama exactamente igual que la clase, no declara tipo de retorno (ni siquiera void) y se ejecuta solo, al usar new.

Si una clase no define ningún constructor, Java le agrega uno sin parámetros que deja los atributos en sus valores por defecto (0, false, null). Eso es lo que permitía escribir new Auto(). Ese constructor regalado desaparece apenas se escribe uno propio: a partir de ahí, new Auto() deja de compilar y hay que pasarle los datos que el constructor pide. Es justamente lo que buscábamos, pero conviene saberlo porque es el error de compilación más común al aplicar este tema por primera vez.

La palabra clave this

Dentro de un método o un constructor, this es una referencia al objeto sobre el que se está ejecutando ese código. Su uso más frecuente es distinguir un atributo de un parámetro cuando los dos se llaman igual, que es lo habitual en los constructores:

```
this.marca = marca;
```

A la izquierda, el atributo del objeto. A la derecha, el parámetro recibido. Sin this, las dos partes se refieren al parámetro y la asignación no hace nada.

Acceso controlado: getters, setters y validación

Con los atributos en private hace falta alguna manera de interactuar con ellos desde afuera, y para eso están los métodos de acceso. Un getter devuelve el valor de un atributo y da acceso de solo lectura: getSaldo() deja ver el saldo, no cambiarlo. Un setter lo modifica, y no es una asignación con más letras: es el lugar donde la clase revisa el dato antes de aceptarlo, y lo rechaza si rompe su promesa.

Cuando el dato es inválido hay que decidir qué hacer. Vamos a seguir la misma convención en toda la materia, y son dos casos distintos.

Un setter no imprime nada. Una billetera no sabe si el programa que la usa corre en una consola, en una ventana o en un servidor, así que no puede decidir cómo avisarle a nadie. Lo que hace es rechazar el valor, dejar el estado como estaba y devolver un boolean que indica si pudo. Quien la llamó es el que sabe cómo mostrar el problema:

```
if (!billetera.setAlias("ab")) {
    System.out.println("Ese alias no es válido.");
}
```

Un constructor no tiene esa salida, porque no puede devolver un boolean para avisar que no pudo: ya está construyendo el objeto. Sus alternativas serían inventar un valor por defecto (pedir una billetera con alias vacío y recibir una llamada “sin alias”, o sea un objeto que miente) o aceptar el dato malo y volver al problema del principio. Por eso, cuando los datos que recibe no permiten construir un objeto válido, el constructor corta la ejecución:

```
throw new IllegalArgumentException("El alias no puede estar vacío.");
```

Esa línea detiene el programa en el punto exacto del error y muestra el mensaje, en lugar de dejar circular un objeto roto. Es una excepción, un mecanismo que varios ya conocen de otros lenguajes; por ahora alcanza con usar esa línea tal cual está.

Un último detalle de sintaxis: un atributo declarado final solo se puede asignar una vez, en el constructor, y después no cambia nunca. Es la forma de decir en el código que hay datos que se fijan al crear el objeto y no se tocan más.

Getters y setters no son un trámite

Los IDE ofrecen generar automáticamente un getter y un setter para cada atributo. Usar ese botón sin pensar es la manera más común de escribir código que parece encapsulado y no lo está: si todos los atributos tienen setter público, el objeto quedó tan expuesto como antes, solo que con el triple de líneas.

La diferencia se ve comparando dos formas de pagar mil pesos:

```
billetera.setSaldo(billetera.getSaldo() - 1000);
billetera.pagar(1000);
```

En la primera, quien decide si la operación es posible es el que llama, y puede olvidarse de revisar el saldo o dejarlo negativo. En la segunda decide la billetera, que es la única que conoce sus reglas. Además la segunda dice lo que pasa: la primera es aritmética sobre un número, la segunda es un pago.

Por eso la pregunta al diseñar una clase no es qué atributos tiene y qué getter y setter le toca a cada uno, sino qué operaciones ofrece. Cada setter se justifica solo si tiene sentido que ese dato cambie desde afuera, y muchos atributos no lo necesitan.

Todo junto

```java
public class Billetera {

    private final String cvu;   // se fija al crear la billetera y no cambia nunca
    private String alias;
    private double saldo;

    public Billetera(String cvu, String alias) {
        if (cvu == null || cvu.length() != 22) {
            throw new IllegalArgumentException("CVU invalido.");
        }
        if (alias == null || alias.length() < 6) {
            throw new IllegalArgumentException("Alias demasiado corto.");
        }
        this.cvu = cvu;
        this.alias = alias;
        this.saldo = 0;         // toda billetera nace en un estado conocido
    }

    public String getCvu() {
        return this.cvu;
    }

    public String getAlias() {
        return this.alias;
    }

    public double getSaldo() {
        return this.saldo;      // se puede consultar, no asignar
    }

    public boolean setAlias(String alias) {
        if (alias == null || alias.length() < 6) {
            return false;       // el alias anterior queda intacto
        }
        this.alias = alias;
        return true;
    }

    public boolean cargar(double monto) {
        if (monto <= 0) {
            return false;
        }
        this.saldo += monto;
        return true;
    }

    public boolean pagar(double monto) {
        if (monto <= 0 || monto > this.saldo) {
            return false;       // la billetera nunca queda en negativo
        }
        this.saldo -= monto;
        return true;
    }
}
```

Tres atributos y tres tratamientos distintos, ninguno casual. El cvu se puede consultar, pero no existe setCvu porque un CVU no cambia. El alias se puede consultar y cambiar, pero solo por

un setter que verifica el nuevo valor. El saldo se puede consultar y no tiene setter en ningún caso: se mueve únicamente con cargar y pagar, que son las operaciones que la billetera realmente ofrece. Desde afuera no hay forma de dejar una billetera sin CVU, con un alias de dos letras o con saldo negativo, por más distraído que esté quien la use. Esa es toda la idea del encapsulamiento.

Para pensar antes de la clase

Con el código de la clase 1 a mano, tres preguntas para traer pensadas:

1. Encontrar tres maneras distintas de dejar un objeto Auto en un estado que no tiene sentido.
2. Escribir la promesa de la clase Auto: qué tiene que ser cierto para que un auto sea válido. En particular, qué valores de anio se aceptan.
3. Decidir cuáles de los tres atributos merecen un setter público y cuáles no, y por qué.

En la práctica de esta semana empieza a tomar forma el proyecto integrador, y cada clase del dominio que aparezca ahí va a tener que responder las mismas tres preguntas.

Bibliografía sugerida para profundizar

- Deitel, P., & Deitel, H. (2017). Java How to Program, Early Objects.
    - Capítulo 3: Introduction to Classes, Objects, Methods and Strings.
    - Capítulo 8: Classes and Objects: A Deeper Look. Es el capítulo específico del tema: control de acceso, this, constructores y atributos final.
- Azpitarte, R. et al. (2009). Introducción a la Programación Orientada a Objetos con Java.
    - Capítulo 3: clases, constructores, modificadores de acceso y encapsulamiento.
- Barraza, O. et al. (2006). Introducción a la programación orientada a objetos.
    - Capítulo 3: estructura fundamental de un programa orientado a objetos.